Python y primeros objetos

Cuaderno de ejercicios

Una secuencia de iniciación para el módulo de Programación orientada a objetos. Aprenderás lo básico de Python mientras utilizas objetos y, después, construyes tus propias clases.

La idea de trabajo

Primero usarás cadenas, listas y una clase Libro preparada. Después de trabajar decisiones, bucles y funciones, escribirás tu propia clase. Cada ejercicio propone anticipar un resultado, ejecutar el código, modificarlo y explicar qué ha ocurrido.

Cómo utilizar el cuaderno

Trabaja un archivo eNN.py por ejercicio. Los archivos iniciales están en la carpeta alumnado. Lee sus requisitos previos antes de empezar.

Resuelve la tarea común y comprueba los resultados indicados. Después intenta la ampliación si puedes explicar tu solución sin ayuda.

Entrega el código, un ejemplo que funcione y una explicación de dos o tres frases. Cuando haya reglas, añade también un caso rechazado.

Los tiempos orientan la práctica; pedir ayuda o necesitar más tiempo forma parte del aprendizaje. Los ejercicios 19 y 20 son opcionales.

Preparación

Python 3.10 o posterior y un editor. No se requieren paquetes externos. Antes del ejercicio 01, dedica unos minutos a abrir, guardar y ejecutar un archivo .py. En una terminal se puede usar python e01.py, python3 e01.py o py e01.py, según la instalación.

Recorrido propuesto

Bloque	Ejercicios	Práctica	Puente hacia objetos
A Primer contacto	01 a 04	70 min	Valores, tipos y métodos de cadenas
B Usar objetos	05 a 07	65 min	Instancias, atributos, estado y condiciones
C Repetir y organizar	08 a 11	110 min	Listas, bucles, funciones y objetos como argumentos
D Construir objetos	12 a 15	125 min	Clase propia, métodos, identidad y colecciones
E Integrar	16 a 18	120 min	Composición, validación y aplicación de consola
F Opcional	19 y 20	70 min	Persistencia JSON y polimorfismo

Tiempo de práctica común: 490 minutos, unas 8 h 10 min. Las ampliaciones de cada ficha no están incluidas. Los ejercicios opcionales suman 70 minutos más.

Vocabulario antes de usarlo

Variable: un nombre asociado a un valor. Atributo: un dato de un objeto. Método: una operación que se llama sobre un objeto. Clase: una definición con la que creamos instancias. Instancia: un objeto concreto de una clase. Estado: los valores que tiene un objeto en un momento dado.

La clase preparada

En los ejercicios 05, 07 y 11, copia modelos_inicio.py a la carpeta de trabajo. Esta es su interfaz de uso; no necesitas escribir todavía su implementación.

```
from modelos_inicio import Libro
libro = Libro("Dune")
print(libro.titulo)          # Consulta un atributo
print(libro.resumen())      # Devuelve un texto
print(libro.prestar())      # True una vez, después False
```

Este primer modelo recibe solo el título. En el ejercicio 12 diseñaremos uno propio con código y título para distinguir ejemplares. No mezcles los dos inicializadores.

01 La ficha de un libro

A Primer contacto · 15 min orientativos · Ruta común

Antes de empezar Ninguno. Saber abrir y ejecutar un archivo .py.

Tarea Crea titulo = "Dune", paginas = 412 y disponible = True. Muestra cada valor y su tipo con print y type. Cambia solo el título a "1984" y vuelve a mostrarlo. Antes de ejecutar, predice qué línea cambiará.

Comprueba Los tipos son str, int y bool. El cambio del título no modifica las páginas ni la disponibilidad.

Pista Una asignación usa =. Un texto lleva comillas. print(titulo) muestra el valor asociado al nombre.

Si avanzas más rápido Añade autor = "Frank Herbert" y comprueba su tipo.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

02 Un cálculo de existencias

A Primer contacto · 15 min orientativos · Ruta común

Antes de empezar Ejercicio 01.

Tarea Una biblioteca tiene 12 ejemplares y ha prestado 5. Calcula cuántos quedan y qué porcentaje representan. Guarda ambos resultados y muéstralos con un texto. Usa una cadena f para incluir valores, por ejemplo f"Quedan {restantes}". Supón que el total es positivo.

Comprueba Quedan 7 ejemplares. El porcentaje es aproximadamente 58,33. Si prestados vale 0, quedan 12 y el porcentaje es 100.

Pista Calcula restantes = total - prestados y porcentaje = restantes / total * 100. No redondees el dato antes de usarlo.

Si avanzas más rápido Calcula cuántas cajas de 5 ejemplares se llenan con los 12 y cuántos sobran.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

03 Datos que escribe una persona

A Primer contacto · 20 min orientativos · Ruta común

Antes de empezar Ejercicios 01 y 02.

Tarea Pide un título y un número de páginas con input. Convierte las páginas con int y muestra cuántas tendría el libro si se añadieran 10. En esta primera versión la entrada numérica será un entero válido. Anota qué ocurre al introducir "muchas" y conserva esa duda para el ejercicio 17.

Comprueba Con Dune y 412, la nueva cantidad es 422. input devuelve texto incluso cuando escribes dígitos. "muchas" produce ValueError en esta versión.

Pista paginas = int(input("Páginas: ")). El texto del mensaje se muestra antes de leer.

Si avanzas más rápido Muestra cuánto ocuparían dos ejemplares si sumamos sus páginas.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

04 Los textos tienen métodos

A Primer contacto · 20 min orientativos · Ruta común

Antes de empezar Variables y cadenas.

Tarea Parte de titulo = " dune ". Predice y ejecuta titulo.strip() y titulo.upper(). Guarda el resultado de strip en limpio y el de upper aplicado a limpio en mayusculas. Muestra los tres nombres con repr para ver sus espacios. No encadenes todavía las llamadas.

Comprueba titulo mantiene sus espacios y minúsculas. limpio vale "dune" y mayusculas vale "DUNE".

Pista El punto permite pedir una operación a un objeto. strip quita espacios en los extremos. upper crea una versión en mayúsculas.

Si avanzas más rápido Compara los títulos " Dune " y "dune" ignorando espacios exteriores y mayúsculas.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

05 Dos objetos de una clase preparada

B Usar objetos · 20 min orientativos · Ruta común

Antes de empezar Ejercicios 01 y 04. El docente entrega modelos_inicio.py.

Tarea Guarda modelos_inicio.py en la misma carpeta. Escribe from modelos_inicio import Libro. Crea a = Libro("Dune") y b = Libro("1984"). Muestra a.titulo y los resultados de a.resumen() y b.resumen(). Explica qué comparten y qué distingue a los dos libros.

Comprueba Los dos son de la clase Libro, pero sus títulos son distintos. Ambos comienzan disponibles.

Pista Libro("Dune") crea una instancia. a.titulo consulta un atributo. a.resumen() llama a un método. No hace falta abrir la clase aún.

Si avanzas más rápido Crea un tercer libro titulado "Fundación" y muestra su resumen.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

06 Una decisión sobre el plazo

B Usar objetos · 20 min orientativos · Ruta común

Antes de empezar Comparación == y operadores < y >, explicados antes de empezar.

Tarea Usa dias = 3 como número de días que faltan para devolver un libro. Si dias es mayor que 0, muestra "En plazo". Si es 0, "Vence hoy". Si es negativo, "Con retraso". Prueba también 0 y -2 cambiando la variable.

Comprueba 3 produce En plazo, 0 produce Vence hoy y -2 produce Con retraso. Solo se ejecuta una rama.

Pista Cada condición termina en dos puntos. El bloque siguiente lleva cuatro espacios.

Si avanzas más rápido Para un retraso, muestra además cuántos días han pasado desde el vencimiento.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

07 El primer préstamo

B Usar objetos · 25 min orientativos · Ruta común

Antes de empezar Ejercicios 05 y 06.

Tarea Crea un Libro de la clase preparada. Llama a prestar dentro de un if. Muestra "Préstamo realizado" si devuelve True y "No disponible" en caso contrario. Repite el intento sobre el mismo objeto. Muestra su resumen después de cada intento.

Comprueba El primer intento tiene éxito y el segundo falla. El libro termina prestado. El método devuelve un booleano y también cambia su estado cuando puede prestar.

Pista if libro.prestar(): llama al método una vez y usa su resultado. No lo llames otra vez solo para imprimir el resultado.

Si avanzas más rápido Crea otro ejemplar con el mismo título y comprueba que sigue disponible.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

08 Una lista que recorre el catálogo

C Repetir y organizar · 25 min orientativos · Ruta común

Antes de empezar Ejercicios 04 y 06.

Tarea Crea titulos = ["Dune", "1984"]. Añade "Fundación" con append. Recorre la lista con for y muestra un título por línea con su número, empezando por 1. Usa un contador que aumente en cada vuelta.

Comprueba Aparecen tres títulos numerados 1, 2 y 3. La lista original sí cambia al llamar a append.

Pista for titulo in titulos: toma un elemento en cada vuelta. numero = numero + 1 aumenta el contador.

Si avanzas más rápido Cuenta cuántos títulos contienen la letra "a", ignorando mayúsculas.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

09 Un menú que se repite

C Repetir y organizar · 25 min orientativos · Ruta común

Antes de empezar input, if y comparación de cadenas.

Tarea Pide una opción de texto mientras no sea "0". "1" muestra "Catálogo". "0" muestra "Hasta luego" y termina. Cualquier otro texto muestra "Opción no válida". No conviertas la entrada a número.

Comprueba Con la secuencia 7, 1, 0 hay un aviso, un Catálogo y una despedida. El programa termina tras 0.

Pista Empieza con opcion = "". Dentro del while vuelve a leer la opción en cada vuelta.

Si avanzas más rápido Cuenta cuántas veces se ha elegido la opción 1 y muéstralo al salir.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

10 Una función que calcula

C Repetir y organizar · 30 min orientativos · Ruta común

Antes de empezar Variables, operaciones y if.

Tarea Define dias_restantes(plazo, transcurridos). Debe devolver la resta, sin imprimirla ni cambiar los argumentos. Llama a la función y muestra el resultado desde fuera. Prueba (14, 4), (14, 14) y (14, 17).

Comprueba Los resultados son 10, 0 y -3. return entrega un valor que se puede guardar. print solo lo muestra.

Pista Una definición comienza con def y un bloque indentado. El cuerpo termina con return plazo - transcurridos.

Si avanzas más rápido Define estado_plazo(plazo, transcurridos), reutilizando dias_restantes, que devuelva En plazo, Vence hoy o Con retraso.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

11 Una función recibe un objeto

C Repetir y organizar · 30 min orientativos · Ruta común

Antes de empezar Ejercicios 05, 07 y 10.

Tarea Define etiqueta(libro). Devuelve "Dune [disponible]" o "Dune [prestado]" según el título y estado del objeto recibido. Crea un Libro preparado y muestra su etiqueta antes y después de prestarlo. La función etiqueta no debe modificarlo.

Comprueba La etiqueta cambia después del préstamo. El objeto mantiene el mismo título. Consultarlo no lo presta.

Pista Dentro de la función puedes usar libro.titulo y libro.disponible. El parámetro recibe el objeto que pasas en la llamada.

Si avanzas más rápido Define prestar_y_etiquetar(libro), que intente prestarlo y devuelva su etiqueta. Explica qué función consulta y cuál modifica.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

12 Nuestra primera clase

D Construir objetos · 30 min orientativos · Ruta común

Antes de empezar Ejercicios 05 y 10.

Tarea Crea tu propia clase Libro. Su inicializador recibe codigo y titulo, guarda ambos en self y establece disponible = True. Crea L1 con Dune y L2 con 1984. Muestra sus títulos. En esta versión añadimos un código para distinguir ejemplares incluso si comparten título.

Comprueba L1 y L2 tienen códigos y títulos independientes. Ambos empiezan disponibles. Cambiar el título del primero no cambia el segundo.

Pista class Libro: contiene def __init__(self, codigo, titulo):. self.codigo = codigo guarda el dato en esa instancia.

Si avanzas más rápido Añade autor al inicializador y crea dos objetos con autores diferentes.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

13 Métodos que cumplen una regla

D Construir objetos · 35 min orientativos · Ruta común

Antes de empezar Clase del ejercicio 12, if, funciones y return.

Tarea Añade prestar y devolver a tu clase. prestar devuelve True y cambia disponible a False solo si estaba disponible. En caso contrario devuelve False y conserva el estado. devolver hace la operación inversa y también informa con un booleano. Usa la clase en cuatro llamadas: prestar, prestar, devolver, devolver.

Comprueba Las llamadas devuelven True, False, True, False. El estado final es disponible. Un intento rechazado no altera el objeto.

Pista En prestar, trata primero el caso if not self.disponible: return False. Después realiza el cambio válido.

Si avanzas más rápido Añade resumen, equivalente a etiqueta del ejercicio 11 pero sin parámetro libro: usa self.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

14 Dos nombres o dos objetos

D Construir objetos · 25 min orientativos · Ruta común

Antes de empezar Clase del ejercicio 13 y asignación.

Tarea Crea a = Libro("L1", "Dune"), b = a y c = Libro("L2", "Dune"). Antes de ejecutar b.prestar(), predice qué ocurrirá con a.disponible y c.disponible. Ejecuta y muestra a is b y a is c. is comprueba si dos referencias apuntan al mismo objeto.

Comprueba a y b nombran el mismo objeto, que queda prestado. c es otro y sigue disponible. a is b es True y a is c es False.

Pista Asignar b = a no crea un ejemplar nuevo. Cada llamada a Libro(...) sí crea una instancia distinta.

Si avanzas más rápido Crea dos listas, copia = lista y separada = lista.copy(). Añade un elemento a copia y explica cuál de las dos listas cambia. La copia es superficial.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

15 Buscar entre objetos

D Construir objetos · 35 min orientativos · Ruta común

Antes de empezar Ejercicios 08, 10 y 13.

Tarea Crea una lista con L1 Dune y L2 1984. Define buscar(libros, codigo): devuelve el objeto cuyo código coincide o None si no existe. Prueba L1 y L9. Solo llama a prestar cuando el resultado no sea None.

Comprueba Buscar L1 devuelve el propio objeto almacenado. Buscar L9 devuelve None. Prestar el resultado de L1 modifica el objeto de la lista.

Pista El return None debe quedar fuera del for. if encontrado is not None: evita pedir métodos a None.

Si avanzas más rápido Define disponibles(libros) que construya y devuelva una lista con los libros disponibles.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

16 Una biblioteca contiene libros

E Integrar · 40 min orientativos · Ruta común

Antes de empezar Ejercicios 13 y 15.

Tarea Define Biblioteca con self.libros = [] en __init__. Añade buscar(codigo), agregar(libro) y prestar(codigo). agregar rechaza códigos repetidos y devuelve un booleano. prestar busca el objeto y delega en su método prestar. Si no lo encuentra, devuelve False. Un mismo título con códigos distintos sí se admite.

Comprueba Agregar L1 funciona una vez. Agregar otro L1 falla sin reemplazar el anterior. Prestar L9 falla. Dos bibliotecas nuevas no comparten su lista.

Pista Biblioteca tiene libros: no hereda de Libro. En los métodos puedes reutilizar self.buscar(codigo).

Si avanzas más rápido Añade total_disponibles que devuelva cuántos libros disponibles tiene la biblioteca.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

17 Una entrada incorrecta no termina el programa

E Integrar · 30 min orientativos · Ruta común

Antes de empezar Ejercicios 03, 09 y 10.

Tarea Define pedir_entero(mensaje). Repite input hasta recibir un entero mayor o igual que cero. Si int falla, muestra "Escribe un entero". Si el valor es negativo, muestra "No puede ser negativo". Si es válido, devuélvelo. Introduce abc, -2, 0.

Comprueba Hay dos avisos y la función devuelve 0. Una entrada vacía o "2.5" también debe volver a pedir el dato.

Pista try rodea la conversión. except ValueError captura ese problema concreto. continue vuelve al inicio del while.

Si avanzas más rápido Define pedir_opcion(mensaje, maximo) que reutilice pedir_entero y rechace valores superiores al máximo.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

18 La biblioteca de consola

E Integrar · 50 min orientativos · Ruta común

Antes de empezar Ejercicios 13, 16 y 17.

Tarea Prepara una Biblioteca con L1 Dune y L2 1984. Implementa un menú: 1 listar, 2 prestar, 3 devolver y 0 salir. Reutiliza pedir_entero. Para préstamo o devolución solicita un código y distingue libro inexistente de operación rechazada. La interfaz muestra mensajes, pero cambia el estado llamando a métodos. Puedes copiar tus clases y funciones en el mismo archivo.

Comprueba No debe fallar al introducir abc como opción. Rechaza prestar dos veces L1 y devolver dos veces L1. L9 se informa como inexistente. Al salir termina el bucle. Los datos se pierden al reiniciar: todavía no hay persistencia.

Pista Primero consigue listar y salir. Después añade préstamo y devolución. No mezcles a la vez todas las opciones.

Si avanzas más rápido Añade la opción 4 para listar solo los disponibles.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

19 Guardar y recuperar el catálogo

F Ampliaciones opcionales · 35 min orientativos · Opcional

Antes de empezar Ejercicio 18. Minilección previa de diccionarios, import, with y JSON.

Tarea Extensión opcional hacia RA3. Define guardar(biblioteca, ruta) y cargar(ruta). Guarda una lista de diccionarios con codigo, titulo y disponible. Al cargar, crea una nueva Biblioteca y nuevos Libro, recuperando el estado mediante prestar si corresponde. En este ejercicio el JSON tiene el formato esperado y códigos únicos. Préstale L1, guarda, carga y comprueba su estado.

Comprueba La nueva biblioteca conserva títulos, códigos y disponibilidad, pero contiene instancias nuevas. Un título con tildes se conserva. JSON almacena datos, no los métodos del objeto.

Pista Un diccionario agrupa pares clave: valor. Usa json.dump y json.load dentro de with open(..., encoding="utf-8").

Si avanzas más rápido Si el fichero no existe, devuelve una biblioteca vacía. Un JSON dañado debe seguir mostrando su error, sin ocultarlo.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

20 La misma llamada con respuestas distintas

F Ampliaciones opcionales · 35 min orientativos · Opcional

Antes de empezar Métodos y clases propios consolidados. Minilección previa de herencia.

Tarea Extensión opcional de RA2. Crea Material con titulo y descripcion(), que devuelve el título. LibroPapel y Revista heredan de Material y redefinen descripcion para anteponer "Libro: " y "Revista: ". Recorre una lista con un objeto de cada tipo y llama a descripcion sin comprobar su tipo.

Comprueba La misma llamada produce "Libro: Dune" y "Revista: Ciencia". El bucle no conoce los detalles de cada clase.

Pista class Revista(Material): hereda el inicializador. Su propio def descripcion(self): sustituye ese comportamiento.

Si avanzas más rápido Añade AudioLibro y comprueba que funciona en el mismo bucle sin modificarlo.

Tu explicación Predicción, resultado observado y una frase que explique la diferencia.

Referencias y continuidad

Los ejercicios, ejemplos y tiempos de esta propuesta son originales y orientativos. Los resultados de aprendizaje oficiales se consultan en el anexo I del Real Decreto 566/2024. La batería prepara el arranque de 5100 y no sustituye la programación del módulo.

Currículo del módulo 5100

https://www.boe.es/diario_boe/txt.php?id=BOE-A-2024-12503

Python números cadenas y listas

https://docs.python.org/es/3/tutorial/introduction.html

Python control de flujo y funciones

https://docs.python.org/es/3/tutorial/controlflow.html

Python clases y objetos

https://docs.python.org/es/3/tutorial/classes.html

Python entrada salida y JSON

https://docs.python.org/es/3/tutorial/inputoutput.html

Python errores y excepciones

https://docs.python.org/es/3/tutorial/errors.html

El siguiente paso

Ampliar el diseño de clases, la organización en módulos y paquetes y las pruebas. Después, profundizar en ficheros y conectar la lógica de la aplicación a una base de datos y una interfaz web. Se conservará la misma separación: la interfaz recoge entradas, los objetos aplican reglas y la persistencia conserva los datos.